



hes. so business.

63-51 Emerging Technologies

Elm

Bregy Noah, Donnet-Money Mégane, Meichtry Micha



Table of Content

- GitHub Structure & Project Flow
- Part 1: What Is Elm and Why Does It Exist?
- Part 2: How Does Elm Compare?
- Part 3: Elm in Practice (Examples)
- Part 4: Prototype Demo
- Part 5: When Would You Actually Use Elm?

GitHub Structure & Project Flow

```

docs/
├── 00-setup/                                # Environment setup and installation
│   └── 01-environment-setup.md
├── 01-project-overview/                    # Project goals and repo navigation
│   └── 01-project-overview.md
├── 02-elm-theory/                          # What Elm is and how it works
│   ├── 01-what-is-elm.md
│   ├── 02-core-concepts.md
│   ├── 03-the-elm-architecture.md
│   └── 04-elm-ecosystem.md
├── 03-comparison/                         # Elm compared to other technologies
│   ├── 01-comparison-overview.md
│   ├── 02-elm-vs-javascript.md
│   ├── 03-elm-vs-typescript-frameworks.md
│   ├── 04-elm-vs-other-functional-options.md
│   ├── 05-when-elm-is-a-good-choice.md
│   ├── 06-when-elm-is-not-a-good-choice.md
│   └── 07-summary.md
├── 04-prototype/                          # Documentation of the prototype
│   ├── 01-prototype-overview.md
│   ├── 02-data-model.md
│   ├── 03-messages-and-update.md
│   ├── 04-view.md
│   └── 05-test-scenarios.md
├── examples/                              # Small code examples
│   ├── 01-counter/                        # Elm Architecture
│   ├── 02-traffic-light/                  # Custom types + pattern matching
│   ├── 03-temperature-converter/          # Maybe + form input
│   ├── 04-ports-localstorage/             # JS interop trade-off
│   └── 05-weather-app/                    # External data sources
└── 99-resources/                          # Research sources and further reading
    └── 01-resources.md
  
```

```

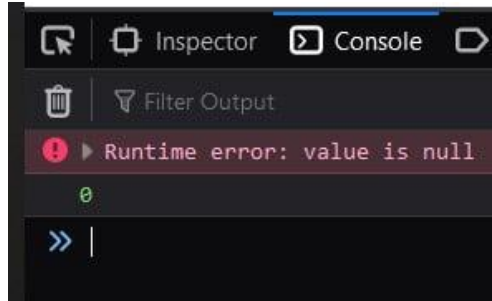
└── prototype/                             # The working Elm application
    ├── README.md                         # How to install and run the prototype
    ├── elm.json
    ├── package.json
    ├── public/
    │   └── index.html
    └── src/
        ├── Main.elm
        ├── Model.elm
        ├── Msg.elm
        ├── Types.elm
        ├── Update.elm
        └── View.elm
  
```



Part 1: What Is Elm and Why Does It Exist?

The Problem

- JavaScript not designed for large, complex frontend applications
- Runtime errors (only appear when code runs)
- Unpredictable state (mutable values and side effects)
- Ecosystem fragility requires stacking React + Redux + TypeScript + Babel ...
- Example:



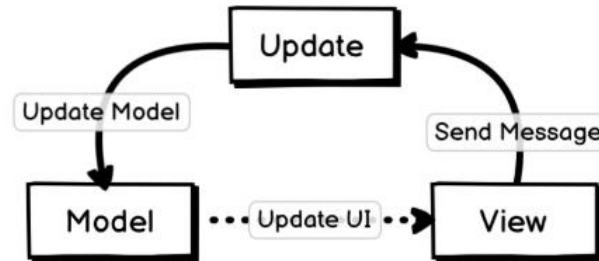
What Elm is

- Functional programming language that compiles to JavaScript
- Designed exclusively for browser-based user interfaces
- Created by Evan Czaplicki as his Harvard thesis in 2012
- Reliability is built into the language, not added on top
- No runtime exceptions by design
 - The compiler catches errors before the program runs



The Elm Architecture (TEA)

- The mandatory pattern every Elm application is built on
 - **Model**: The complete application state - one place, no hidden values
 - **Msg**: Every possible user action declared as a custom type variant
 - **Update**: Pure function: takes Model + Msg, returns a new Model
 - **View**: Pure function of the Model - same input always produces same HTML



The Key Guarantees

- **No null, no undefined, no exceptions**
 - Absent values are expressed as Maybe, which the compiler forces you to handle
- **Exhaustive pattern matching**
 - Every case expression must handle all variants - missing branches are compile errors
- **Immutability**
 - Update always returns a new Model - the old one is never mutated
- **Friendly compiler errors**
 - The compiler names the missing variant and tells you what to do next



Friendly Compiler Messages

TYPE MISMATCH

[jump to error](#)

The type annotation for `greeting` does not match its definition.

```
3 | greeting : Int -> String
```

The type annotation is saying:

```
Int -> String
```

But I am inferring that the definition has this type:

```
String -> String
```

- Type annotation does not match function definition
- Annotation: `Int -> String`
- Actual: `String -> String`
- Cause: Function treats input as `String`
- Fix: Change annotation to `String -> String`



Security

Handled by Design

- XSS via DOM injection «Html.text»
 - Escapes all output automatically
- No raw HTML injection possible from code
- No dynamic code execution at runtime
 - Compiled to static JavaScript
- Type-safe form inputs → no unvalidated strings reach the logic
- Package logic of Elm Package registry
 - Code on GitHub is reviewable

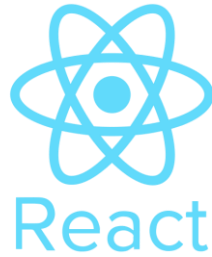
Outside Elm'Scope

- Port boundary, JS subscribers not checked
- npm building tool → third-party packages
- Network security (HTTPS, CORS ...)
- Authentication and authorization
 - Needs to be done by backend
- Backend and API security

Security in Elm

Part 2: How Does Elm Compare?

The Comparison Landscape

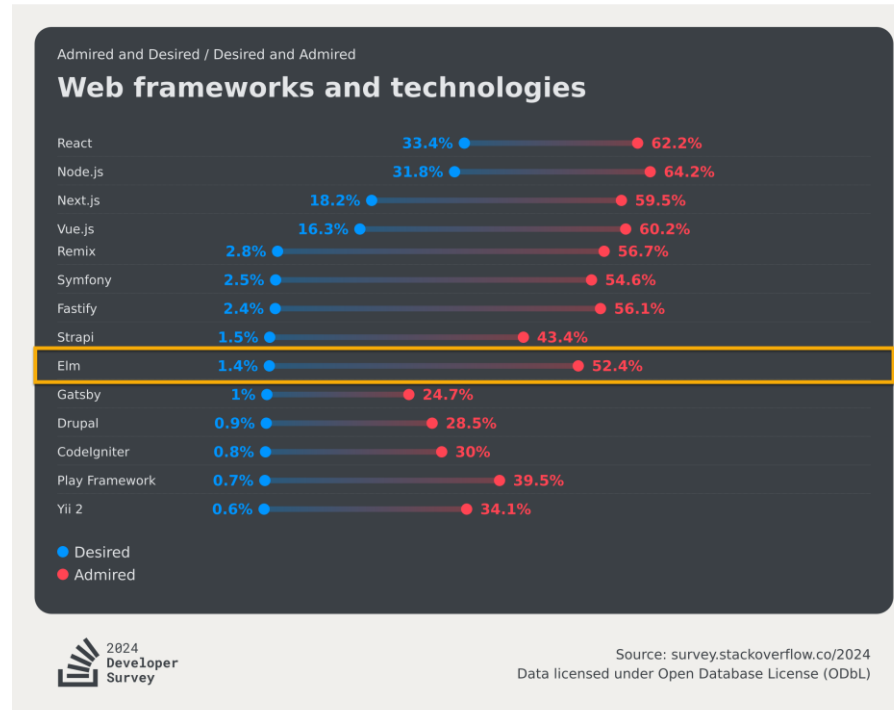


The Key Trade-off in One Table

Technology	Type safety	Runtime errors	Architecture	Ecosystem	JS interop	Team suitability
JavaScript	None	Common class of errors	None enforced	Very large	Direct	Universal
TypeScript	Partial, opt-out available	Reduced, not eliminated	None enforced	Very large	Direct	Very common
React + TypeScript	Partial, opt-out available	Reduced, not eliminated	None enforced	Very large	Direct	Large hiring pool
Vue + TypeScript	Partial, opt-out available	Reduced, not eliminated	Conventions, not enforced	Large	Direct	Good hiring pool
ReScript	Very high, seam at JS FFI	Eliminated in pure code	None enforced	Full npm access	Direct via FFI	Easier with React background
PureScript	Maximum, Haskell-style	Eliminated in pure code	None enforced	Small	Two-file FFI	Rare in commercial use
Elm	Full, no opt-out	Eliminated by design	TEA enforced	Small, curated	Through ports only	Specialist knowledge required

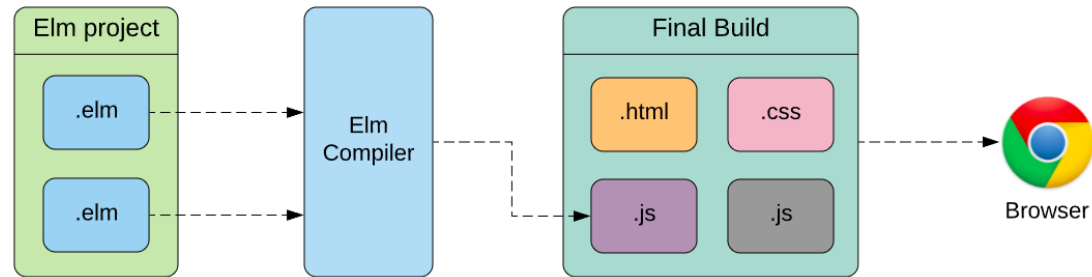
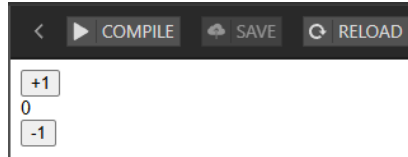


The Admiration Gap (Stack Overflow 2024)



Part 3: Elm in Practice (Examples)

How to Run Elm Code



```

1 npm install -g elm
2
3 elm make src/Main.elm --output=public/app.js
  
```



Example 1-5

Example 1: Counter - The Elm Architecture in under 50 lines

Example 2: Traffic Light - Custom types and exhaustive pattern matching

Example 3: Temperature Converter - Maybe type and input validation

Example 4: Persistent Note - Ports and JavaScript interop (localStorage)

Example 5: Weather App - HTTP requests and JSON decoding



Example 1: Counter

A counter with three buttons: +, -, Reset

Model: A single Int - the entire application state

Msg: Increment, Decrement, Reset

Update: case msg of - each branch returns a new Int, nothing is mutated



Example 2: Traffic Light

A traffic light cycling Red, Green, Yellow, Red

Custom type:

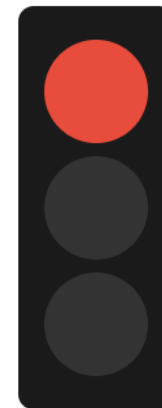
```
type TrafficLight = Red | Yellow | Green
```

invalid states are impossible

Pattern matching:

```
update, lightColor, and label
```

all case-match on every variant independently



Red - Stop

Next



Part 4: Prototype Demo

Prototype Description

- **IT Service Desk Ticket Management System**
 - Built entirely in Elm, runs in the browser with no backend
- **Why this domain?**
 - Tickets, statuses, filters, and forms map naturally onto TEA
- **Features**
 - Create tickets, filter by status, search by keyword, view details, post comments, track history
- **Files**
 - Main.elm / Types.elm / Model.elm / Msg.elm / Update.elm / View.elm



Prototype Demo

IT Service Desk
Ticket Management System

All (4) Open (1) In Progress (1) Resolved (1) Closed (1)

Search tickets... + New Ticket

#1 Open High Due 2026-04-25
Cannot connect to VPN
Network
Open View Details

#2 In Progress Critical Due 2026-04-28
Outlook crashes on startup
Software
In Progress View Details

#3 Resolved Low
Request new keyboard
Hardware
Resolved View Details

#4 Closed Medium Due 2026-04-24
Reset domain password
Access
Closed View Details

```

prototype/src/
├─ Main.elm      # Entry point: wires together init, update, and view via Browser.sandbox
├─ Types.elm     # All custom type definitions: TicketStatus, Priority, FilterState, Ticket, TicketComment, TicketHistoryEntry
├─ Model.elm     # The Model record, the init function, and the seed ticket data
├─ Msg.elm      # All Msg variant declarations with documentation comments
├─ Update.elm    # The update function and all state transition logic
└─ View.elm     # All view functions: rendering, filtering, form, detail, comments, history
  
```



What the Prototype Demonstrates (Test Scenarios)

- **Status badges (Open / In Progress / Resolved / Closed)**
 - Custom type - invalid states are structurally impossible
- **Due date badge turns red when overdue**
 - Maybe String - absent due date cannot be accidentally used
- **Form error appears and clears as you type**
 - Maybe String in the model - Nothing renders nothing
- **History timeline grows with each status change**
 - Immutable record update - old ticket replaced, never mutated
- **Filter counts and search update in real time**
 - Pure List.filter on the model - no cached state
- **All state survives navigation between views**
 - Single Model - no component state, no sync needed



Part 5: When Would You Actually Use Elm?

When Elm Is a Good Choice

Condition	Why Elm fits
Complex UI state that must remain predictable over time	TEA enforces a single location for state and a single path for changes
High reliability requirement with low tolerance for runtime failures	Compile-time elimination of null errors, unhandled cases, and type mismatches
Team prepared to invest in a defined learning period	Finite language surface, high admiration rate among actual users, no ongoing architectural decisions
Educational or architecture-driven project	Explicit design, teachable concepts, examples that demonstrate principles directly
Small team where consistency matters more than hiring breadth	Enforced structure means any contributor can read any part of the codebase immediately



When Elm Is Not a Good Choice

Condition	Why Elm does not fit
Application depends heavily on third-party JavaScript libraries	Port overhead accumulates to the point of working against the application
Organisation needs to hire from a broad talent pool	Elm usage at 1.4% means the available pool is narrow and onboarding is slow
Short timeline or throwaway prototype	Front-loaded costs do not pay for themselves without a long-term maintenance benefit
Application built primarily on browser APIs outside Elm's standard packages	Every native API requires port infrastructure, which compounds across the application
Large team with mixed backgrounds and no structured learning investment	Enforced constraints generate friction for developers who have not chosen to adopt them





School of Management
Route de la Plaine 2
3960 Sierre

hevs.ch/heg



Thank you for your attention.

swissuniversities

